

```

1  """
2  Programma monitor.py per il monitoraggio della cartella C:\\Work\\marconilab.
3
4  Il programma registra l'avvio in un file di log e controlla
5  le modifiche che avvengono nella cartella indicata.
6  Ogni modifica viene stampata a video e salvata nel file di log.
7  """
8
9  __author__ = "Carloumbero Olivieri e Simone Volpe"
10 __version__ = "Rev. 1.3 del 2026-05-26"
11
12 from pathlib import Path # Importa Path, utile per gestire percorsi di file e cartelle in modo semplice
13 import time # Importa time, usata per ottenere data e ora da inserire nei log
14 from watchfiles import watch # Importa watch, funzione che controlla le modifiche in una cartella
15
16 def scrivi_log(messaggio, percorso_log): # funzione scrivi_log
17     """
18     Scrive un messaggio nel file di log.
19     Ogni riga del log contiene:
20     - data e ora dell'evento;
21     - descrizione dell'evento.
22     """
23
24     # Crea una stringa con data e ora attuali nel formato anno-mese-giorno ore:minuti:secondi
25     orario = time.strftime("%Y-%m-%d %H:%M:%S")
26
27     # Prepara la riga da scrivere nel log, inserendo orario e messaggio
28     riga = f"[{orario}] {messaggio}\n"
29
30     # Apre il file di log in modalità append, cioè aggiungendo il testo in fondo al file
31     with open(percorso_log, mode="a", encoding="utf-8") as f_log:
32         f_log.write(riga) # Scrive la riga nel file di log
33
34     print(riga, end="") # Stampa la stessa riga anche a video
35
36 def monitora_cartella(cartella_da_osservare, percorso_log): # funzione monitora_cartella
37     """
38     Monitora una cartella e registra nel file di log gli eventi rilevati.
39     Gli eventi potranno in futuro essere usati anche per avviare
40     operazioni di backup.
41     """
42
43     # Mostra a video quale cartella viene monitorata
44     print(f"Monitoraggio avviato su: {cartella_da_osservare}")
45
46     # watch resta in ascolto e restituisce le modifiche rilevate nella cartella
47     for modifiche in watch(cartella_da_osservare):

```

```

48
49
50     # Ogni modifica contiene il tipo di evento e il percorso del file coinvolto
51     for tipo_evento, percorso_file in modifiche:
52         # Prepara il messaggio descrittivo dell'evento rilevato
53         messaggio = f"- Evento: {tipo_evento.name} | File: {percorso_file}"
54
55         # chiama la funzione scrivi_log per scrivere il messaggio nel file di log
56         scrivi_log(messaggio, percorso_log)
57
58 def main(): # funzione main
59     """
60     Stabilisce i percorsi principali, registra l'avvio del programma
61     e avvia il monitoraggio della cartella scelta.
62     """
63
64     # Percorso della cartella da controllare
65     cartella_da_osservare = Path(r"C:\Work\marconilab")
66
67     # Percorso relativo del file di log, costruito partendo dalla posizione di questo file Python
68     cartella_corrente = Path(__file__).parent # Cartella in cui si trova monitor.py
69     cartella_progetto = cartella_corrente.parent # Cartella principale del progetto
70     file_log = cartella_progetto / "log" / "monitor.log" # Percorso completo del file di log
71
72     # Registra nel log l'avvio del programma
73     scrivi_log("Programma avviato", file_log)
74
75     print(f"--- monitor.py attivo su {cartella_da_osservare} ---") # messaggio da stampare a video
76     print("Premi Ctrl + C per fermarlo se sei in console.") # messaggio da stampare a video
77
78     # Avvia il monitoraggio della cartella
79     monitora_cartella(cartella_da_osservare, file_log)
80
81 if __name__ == "__main__":
82     main() #chiama la funzione main

```